

1.What is the purpose of data base system?

The primary purpose of a Database Management System (DBMS) is to store, retrieve, and manage data efficiently, securely, and conveniently. It was developed to overcome the severe limitations of traditional file-processing systems.

- **Data Independence:** A DBMS isolates the underlying data storage details from the application programs, meaning changes in storage structure do not require rewriting the applications.
- **Reduction of Redundancy:** By centralizing data, a DBMS minimizes duplicate data across multiple files, which saves storage space and prevents inconsistency.
- **Data Integrity:** It enforces constraints and rules (like primary keys and data types) to ensure the data entered is accurate and valid.
- **Concurrent Access and Crash Recovery:** A DBMS allows multiple users to access data simultaneously without conflict and provides robust mechanisms to recover data in case of system failure.
- **Security:** It provides authentication and authorization mechanisms to restrict unauthorized access to sensitive information.

2.Describe the three levels of data abstraction?

Data abstraction hides the complex details of how data is actually stored and maintained. The architecture of a database is divided into three distinct levels:

- **Physical Level (Internal Level):** This is the lowest level of abstraction. It describes exactly *how* the data is actually stored on the physical storage media (like hard drives). It deals with complex low-level data structures, indexing, and access paths.
- **Logical Level (Conceptual Level):** This level describes *what* data is stored in the database and the relationships that exist among those data points. Database administrators work at this level to define tables, data types, and logical constraints without worrying about physical storage details.
- **View Level (External Level):** This is the highest level of abstraction, designed for end-users. It describes only a specific part of the entire database relevant to a particular user or application, hiding the rest of the database and its logical complexities.

3.Describe schema and instance in DBMS.

- **Schema:** The schema is the overall logical structure and design of the database. It is defined during the database design phase and is expected to remain relatively static. It acts as a blueprint, detailing the tables, attributes, data types, and constraints. For example, stating that a **STUDENT** table has **Roll_No** (integer) and **Name** (string) is part of the schema.
- **Instance:** An instance is the actual content or snapshot of the data present in the database at a specific moment in time. While the schema is static, the instance changes dynamically every time data is inserted, updated, or deleted. The actual record of "Roll_No: 1, Name: John" currently in the table is an instance.

4.What are the different types of schema?

The types of schemas correspond directly to the three levels of data abstraction in a database architecture:

- **Physical Schema:** Defines the physical storage structures, access methods, and file organization of the database on the disk.
- **Logical Schema:** Defines all logical constraints, tables, views, and integrity rules. Programmers and database administrators rely on the logical schema to understand how data entities relate to one another.
- **View Schema (External Schema):** Defines different views of the database for different user groups. A single database can have multiple view schemas tailored to the specific needs and access privileges of various departments.

5.Explain the database design process in detail.

The database design process is a structured methodology for planning and creating a robust database.

- **Requirements Analysis:** This initial phase involves gathering information from users and stakeholders to understand what data needs to be stored, how it will be used, and the functional requirements of the system.
- **Conceptual Database Design:** Using the gathered requirements, designers create a high-level conceptual data model, most commonly an Entity-Relationship (ER) diagram. This maps out entities, attributes, and relationships without worrying about the specific DBMS being used.
- **Logical Database Design:** The conceptual model is mapped to a specific data model supported by the chosen DBMS (usually the relational model). ER diagrams are converted into a set of relational tables.
- **Schema Refinement (Normalization):** The relational tables are analyzed and restructured to remove data redundancies and potential insertion, update, or deletion anomalies.
- **Physical Database Design:** The logical schemas are translated into actual physical storage structures. Decisions are made regarding indexing, data types, file organization, and storage media to optimize performance.
- **Security and Application Design:** Finally, user roles, access permissions, and application interfaces are developed to ensure secure and efficient interaction with the database.

6.Explain the difference between meta data and data dictionary with example.

- **Metadata** is simply defined as "data about data." It is the descriptive information about the structure, format, and properties of the actual data. For example, knowing that a column named **Salary** must contain numeric values and has a maximum length of 10 digits is metadata.
- **Data Dictionary** (or System Catalog) is the centralized repository or software component within the DBMS that physically stores and manages all this metadata. It contains the definitions of all tables, schemas, user permissions, and constraints.
- **Difference Example:** If a database tracks library books, the rule stating "a book's ISBN must be 13 characters long" is the metadata. The internal system table within the DBMS that saves this 13-character constraint so the database engine can enforce it is the data dictionary.

7.What are the different data models?

Data models determine how data is structured, connected, and processed within the system.

- **Relational Model:** The most widely used model, representing data as two-dimensional tables (relations) consisting of rows and columns.
- **Entity-Relationship (ER) Model:** A conceptual model that represents data as real-world entities and the relationships between them, typically used for database design.
- **Hierarchical Model:** Data is organized in a tree-like structure with a single root, where each child node has exactly one parent node (resembling a parent-child relationship).
- **Network Model:** Similar to the hierarchical model, but it allows a child record to have multiple parent records, forming a complex graph structure.
- **Object-Oriented Model:** Data is represented as objects, similar to object-oriented programming, combining data and methods into single structures.

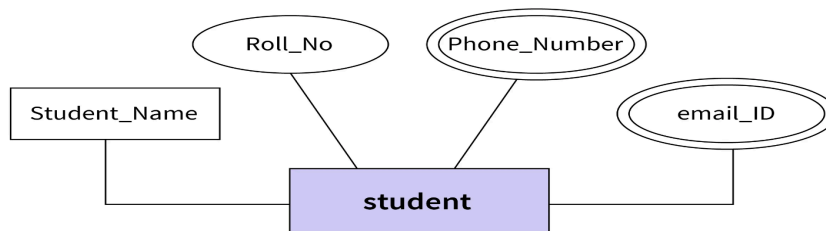
8.Define Entity, Attributes, Entity set, relationship with appropriate notations in ERD?

- **Entity:** An entity is a distinguishable real-world object or concept that exists independently (e.g., a specific Person, Car, or Employee). In an ER Diagram (ERD), an entity is represented by a **Rectangle**.

- **Entity Set:** A collection of entities of the same type that share the same properties or attributes (e.g., all Students in a university). It is also represented by a **Rectangle** named with a plural or collective noun.
- **Attributes:** Properties or characteristics that describe an entity (e.g., Name, Age, Salary). In an ERD, attributes are represented by **Ellipses (Ovals)** connected to their respective entity rectangle.
- **Relationship:** An association or link between two or more entities (e.g., an Employee "Works_In" a Department). In an ERD, a relationship is represented by a **Diamond** connecting the participating entities.

9. Draw the notation for multivalued attributes? Give one example?

Example : ER Model of Student A student can have multiple phone numbers. In this case, the phone number can be classified as a multivalued attribute of the student entity and these multi-valued attributes can be marked inside the co-centric elliptical shape.



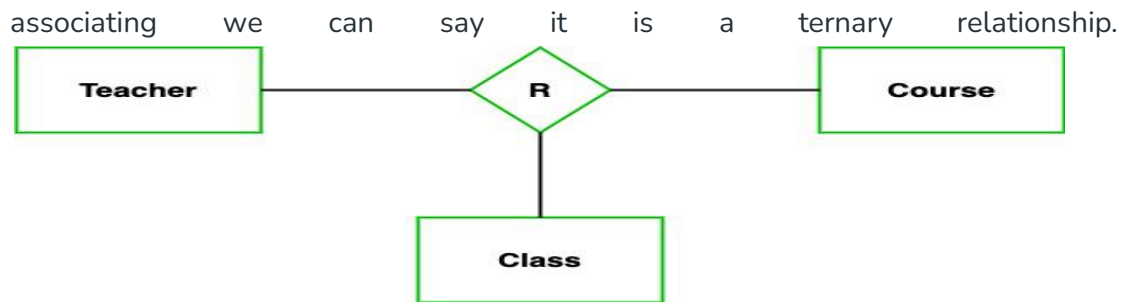
Based on the above illustration, the Student entity has the following four attributes: As previously mentioned, the attributes Student_Name and Role_No are single-valued; however, the attributes Phone_No and Email_ID, which are represented by co-centric ellipses, are multi-valued. We require these attributes to be multi-valued so they may contain several values at once for an entity instance because each student in the real world can submit more than one email address and a phone number.

10. Discuss about Database users and Administrators?

- **Naive Users:** These are unsophisticated users who interact with the system by invoking standard application programs (like bank tellers using graphical software or customers buying items online). They do not write SQL or know the database structure.
- **Application Programmers:** Computer professionals who write backend application programs (using Java, Python, C++, etc.) that interact with the database via DML calls to provide interfaces for naive users.
- **Sophisticated Users/Analysts:** Users who interact with the database without writing applications. They formulate complex queries using query languages (like SQL) to gather data for analysis and business intelligence.
- **Database Administrator (DBA):** The central authority responsible for the entire database system. Duties include defining the schema, granting user access, monitoring performance, performing routine backups, handling disaster recovery, and updating software.

11. Draw ER diagram for Ternary Relationship set with suitable example? What does the cardinality ratio specify?

- **Example:** We have three entity types 'Teacher', 'Course', and 'Class'. The relationship between these entities is defined as the teacher teaching a particular course, also the teacher teaches a particular class. So, here three entity types are



- **Cardinality Ratio:** The cardinality ratio specifies the maximum number of relationship instances that a single entity can participate in. It defines the numerical mapping between entities, such as One-to-One (1:1), One-to-Many (1:N), Many-to-One (N:1), or Many-to-Many (M:N).

12. List any eight applications of DBMS.

1. **Banking:** For managing customer accounts, loans, and executing secure, concurrent financial transactions.
2. **Airlines:** For maintaining flight schedules, seat reservations, and ticketing information.
3. **Universities:** For storing student details, course registrations, grades, and faculty information.
4. **Telecommunications:** For keeping records of calls, generating monthly bills, and managing network infrastructure balances.
5. **E-commerce/Online Retail:** For managing shopping carts, inventory tracking, and customer order history.
6. **Human Resources:** For tracking employee profiles, payroll distributions, tax deductions, and benefits.
7. **Manufacturing:** For supply chain management, tracking production lines, and inventory control.
8. **Healthcare:** For securely storing patient medical histories, billing information, and hospital administrative records.

13. Explain about Database languages with examples?

Database languages are specialized sets of commands used to interact with the DBMS.

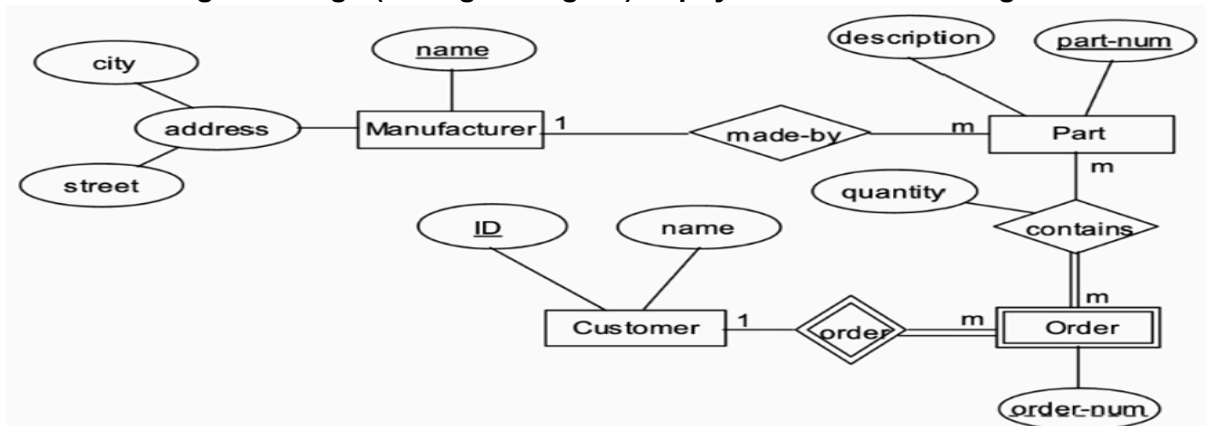
- **Data Definition Language (DDL):** Used to define, alter, or drop the logical structure and schema of the database. Examples: **CREATE TABLE**, **ALTER TABLE**, **DROP TABLE**.
- **Data Manipulation Language (DML):** Used to manipulate the actual data instances within the tables. Examples: **SELECT** (to retrieve data), **INSERT** (to add data), **UPDATE** (to modify data), **DELETE** (to remove data).
- **Data Control Language (DCL):** Used to enforce database security by managing user access privileges. Examples: **GRANT** (to give permissions), **REVOKE** (to take away permissions).
- **Transaction Control Language (TCL):** Used to manage changes made by DML commands and ensure database consistency. Examples: **COMMIT** (to save changes permanently), **ROLLBACK** (to undo changes).

13. Define Relational Database. Explain data representation in RDBMS.

- **Definition:** A Relational Database is a type of database founded on the relational model introduced by E.F. Codd. It organizes data into distinct, independent tables that can be linked—or related—to each other based on common attributes.
- **Data Representation:** In a Relational Database Management System (RDBMS), data is strictly represented as a collection of two-dimensional tables called **relations**.
- **Rows (Tuples):** Each row in a table represents a single, specific, unique record or instance of the entity.
- **Columns (Attributes):** Each column represents a specific property or field of the entity, holding a specific data type.

- **Keys:** Data representation relies heavily on keys (like Primary Keys and Foreign Keys) to uniquely identify records within a table and establish logical connections between different tables.

14. Convert logical design (as in given figure) to physical database design.



Based on the entities, attributes, and relationships extracted from the provided ER diagram, here is the conversion into a physical relational database design (table schemas):

1. Manufacturer Table

- **Attributes:** *name* (Primary Key), *city*, *street*
- **Note:** The composite attribute *address* is broken down into its simple components (*city*, *street*) in the physical table.

2. Part Table

- **Attributes:** *part-num* (Primary Key), *description*, *Manufacturer_name* (Foreign Key)
- **Note:** Because of the 1:M relationship ("made-by") between Manufacturer and Part, the Primary Key of the "1" side (Manufacturer) is placed as a Foreign Key on the "M" side (Part).

3. Customer Table

- **Attributes:** *ID* (Primary Key), *name*

4. Order Table (Weak Entity)

- **Attributes:** *order-num* (Partial Key), *Customer_ID* (Foreign Key)
- **Primary Key:** Composite key of (*order-num*, *Customer_ID*)
- **Note:** The double lines around "Order" and the "order" relationship indicate it is a weak entity dependent on Customer. It cannot exist without a Customer, so its primary key includes the parent's primary key.

5. Contains Table (Mapping the M:N Relationship)

- **Attributes:** *part-num* (Foreign Key), *order-num* (Foreign Key), *Customer_ID* (Foreign Key), *quantity*
- **Primary Key:** Composite key of (*part-num*, *order-num*, *Customer_ID*)

15. Draw the Architecture of Database?

18. Differentiate File Processing System with Database Management System.

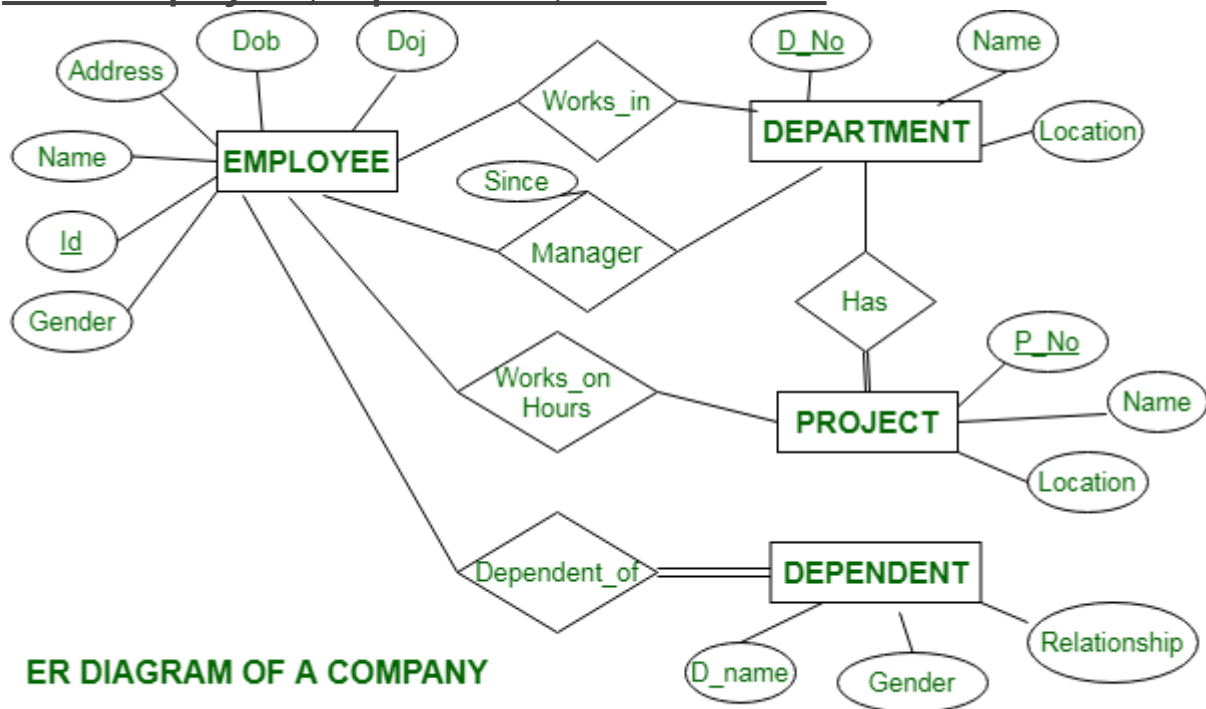
Feature	File Processing System (FPS)	Database Management System (DBMS)
Data Redundancy	High redundancy; identical data is often duplicated across multiple separate files, wasting storage space.	Low redundancy; data is centralized and integrated, drastically reducing duplication.
Data Consistency	Prone to inconsistency. Updating a record in one file might not update it in another.	High consistency. Centralized updates ensure all users see the same, correct data.
Data Isolation	Data is isolated in separate files of potentially varying formats, making it difficult to write new applications to retrieve it.	Data is integrated and formatted uniformly, making access easy and standard.
Data Dependence	Application programs are dependent on the physical file structure. Changes to the file format require rewriting the code.	High data independence. Changes to the physical database structure do not require rewriting applications.
Concurrency	Very difficult to handle concurrent access. Two users trying to edit the same file simultaneously can corrupt it.	Built-in concurrency control mechanisms allow thousands of users to read and write simultaneously without conflicts.
Security	Poor security; usually relies on the operating system's basic file permissions.	Excellent security features, including role-based access control, encryption, and specific privileges.

19. Explain about integrity constraints over relations?

Integrity constraints are strict rules applied to database tables to ensure the accuracy, validity, and consistency of the data over time.

1. **Domain Constraints:** Ensure that the values entered into a specific column adhere to defined data types, formats, or value ranges. For example, an **Age** attribute must be an integer, and logically, should be ≥ 0 .
2. **Key Constraints (Uniqueness Constraints):** Require that a specified attribute (or set of attributes) uniquely identifies each tuple (row) within a relation. Every relation must have at least one candidate key.
3. **Entity Integrity Constraint:** A rule stating that the Primary Key of a relation cannot hold a **NULL** value. If a primary key is null, it is impossible to uniquely identify that specific row.
4. **Referential Integrity Constraint:** This constraint governs the relationships between two tables. It states that if a foreign key exists in a relation, its value must either match an existing primary key value in the referenced relation, or it must be **NULL**. This prevents "orphan" records.

20. Draw the ER diagram for a company needs to store information about employees, departments, and children...



21. Explain the terms with example: Weak Entity set, candidate key, primary key, foreign key

- **Weak Entity Set:** An entity set that does not have sufficient attributes to form a primary key of its own. Its existence depends entirely on a strong entity (the identifying entity).
 - *Example:* An **Insurance_Dependent** entity only exists in the database if the strong **Employee** entity it belongs to exists. If the employee is deleted, the dependent is deleted.
- **Candidate Key:** A minimal set of attributes (one or more) that can uniquely identify a tuple in a relation. A table can have multiple candidate keys. "Minimal" means no subset of the key can be used to uniquely identify the row.
 - *Example:* In a **Student** table, both **Roll_Number** and **Aadhar_Number** (National ID) can uniquely identify a student. Both are candidate keys.
- **Primary Key:** The specific candidate key chosen by the database designer to act as the principal means of identifying rows in the table. It must be strictly unique and cannot contain **NULL** values.
 - *Example:* Out of the candidate keys mentioned above, the university would choose **Roll_Number** as the Primary Key for internal systems.
- **Foreign Key:** An attribute (or group of attributes) in one table that references the primary key of another table. It is the mechanism used to link tables and enforce referential integrity.
 - *Example:* In an **Employee** table, an attribute **Department_ID** is a foreign key that references the **Dept_ID** primary key in the **Department** table.

22. List and Explain set operators of relational algebra.

Set operators in relational algebra take two relations as input and produce a single relation as output. For the first three operators below, the two relations must be **union-compatible** (meaning they must have the same number of attributes, and corresponding attributes must have the same data type).

1. **Union ($\cup$):** The union of two relations R and S ($R \cup S$) returns a relation containing all tuples that appear in R, in S, or in both. Duplicate tuples are automatically eliminated.
2. **Intersection ($\cap$):** The intersection ($R \cap S$) returns a relation containing only the tuples that appear in *both* relation R and relation S.
3. **Set Difference ($-$):** The set difference ($R - S$) returns a relation containing tuples that appear in relation R but do *not* appear in relation S.
4. **Cartesian Product ($\times$):** Also known as the cross product. $R \times S$ combines every tuple from relation R with every tuple from relation S. If R has M rows and S has N rows, the result will have M times N rows. It does not require union compatibility.

23.State the difference between cartesian product and natural join in context of relational algebra.

- **Cartesian Product ($\times$):** This operator combines all rows from the first table with all rows from the second table blindly, regardless of whether the data relates to each other. If Table A has 5 rows and Table B has 4 rows, the output will unconditionally have 20 rows. It does not look for matching columns.
- **Natural Join ($\bowtie$):** This is a much more intelligent and restrictive operation. A natural join only combines rows from two tables if they share the exact same values across all columns that share the same name. Furthermore, it removes the duplicate column in the resulting output. It is essentially a combination of a Cartesian product, followed by a selection (filtering based on matching values in common columns), followed by a projection (removing the redundant column).

24.Give an example of how to use the rename operation in relational algebra.

The rename operation, denoted by the Greek letter Rho (ρ), allows you to change the name of a relation or its attributes. This is critically useful when you need to join a table with itself (a self-join) to avoid ambiguity.

- **Syntax:** $\rho_{\{S\}}(R)$ renames relation R to S. $\rho_{\{S(B_1, B_2)\}}(R)$ renames the relation to S and renames its attributes to B_1, B_2 .
- **Example:** Suppose you have a single table **Employee(EmpID, Name, ManagerID)** where **ManagerID** is a foreign key pointing back to **EmpID**. To find the names of employees alongside the names of their managers, you must join the table with itself. You use ρ to create a renamed copy of the table:

$$\pi_{\{E.Name, M.Name\}}(\rho_E(Employee) \bowtie_{\{E.ManagerID = M.EmpID\}} \rho_M(Employee))$$
 Here, the relation is renamed to E (acting as employees) and M (acting as managers).

25.Explain the following

- i. **Data Definition Language (DDL):** A subset of SQL used to define, alter, and manage the structural layout of database objects (schemas, tables, views, indexes). It deals with the metadata rather than the data itself. Examples include **CREATE**, **ALTER**, **DROP**, and **TRUNCATE**.
- ii. **Data Manipulation Language (DML):** A subset of SQL used to manipulate the actual data contained within the database tables. This includes retrieving, inserting, modifying, and deleting records. Examples include **SELECT**, **INSERT**, **UPDATE**, and **DELETE**.
- iii. **Data Control Language (DCL):** A subset of SQL used to manage database security and access rights. Administrators use DCL to enforce data protection by granting or revoking privileges to specific users. Examples include **GRANT** (give access) and **REVOKE** (remove access).
- iv. **Views:** A view is a virtual table whose contents are defined by a pre-written SQL query. It does not physically store data on the disk (unlike a standard table); instead,

it dynamically pulls data from underlying base tables every time it is queried. Views are used to simplify complex queries and restrict user access to sensitive columns.

26. What are aggregate functions? Explain five built-in aggregate functions.

Aggregate functions in SQL perform a calculation on a set of values across multiple rows and return a single, scalar summarizing value. They are commonly used in conjunction with the **GROUP BY** clause.

1. **COUNT()**: Returns the total number of rows that match a specified criterion. (e.g., **SELECT COUNT(*) FROM Students**; returns the total number of students).
2. **SUM()**: Calculates and returns the total sum of a numeric column. (e.g., **SELECT SUM(Salary) FROM Employee**; returns the total payroll cost).
3. **AVG()**: Calculates and returns the mathematical average value of a numeric column. (e.g., **SELECT AVG(Marks) FROM Exams**;).
4. **MIN()**: Returns the smallest or lowest value within the selected column. (e.g., **SELECT MIN(Price) FROM Products**;).
5. **MAX()**: Returns the largest or highest value within the selected column. (e.g., **SELECT MAX(Age) FROM Patients**;).

27. What is a view? How can it be created? Explain with an example.

As defined previously, a **view** is a virtual, logical table based on the result-set of an SQL query.

How it is created:

It is created using the **CREATE VIEW** DDL statement, followed by the view name, and then the **AS SELECT** statement containing the desired query logic.

Example:

Suppose you have a base table **EMPLOYEE (EmpID, Name, Dept, Salary)**. You want to allow department managers to see their employees' names, but you want to hide the **Salary** column for privacy. You can create a view:

SQL

```
CREATE VIEW HR_Employee_View AS
SELECT EmpID, Name, Dept
FROM EMPLOYEE;
```

Now, users can run **SELECT * FROM HR_Employee_View**; and they will retrieve a table containing only those three columns, effectively hiding the sensitive salary data.

28. What is a join operator? Explain about conditional join and natural join with syntax and example.

A **Join** operator in relational algebra and SQL is used to combine columns from two or more tables into a single result set, based on a related column between them. It is essential for querying normalized databases where data is spread across multiple tables.

- **Conditional Join (Theta Join, denoted as $\bowtie_{\theta}$)**: This join combines tuples from two relations based on *any* specified logical condition (like $<$, $>$, $=$, $\neq$). It does not require columns to have the same name.
 - **Syntax**: **SELECT * FROM TableA JOIN TableB ON TableA.Column X <condition> TableB.Column Y;**
 - **Example**: Finding all products whose price is strictly greater than a specific supplier's raw material cost. **SELECT * FROM Product JOIN Supplier ON Product.Price > Supplier.Material_Cost;**
- **Natural Join (denoted as $\bowtie$)**: This is an automated join that blindly links two tables based on all columns that have the exact same name and data type in both tables. It only returns rows where the values in these common columns are identical, and importantly, it strips out the duplicate columns from the result.
 - **Syntax**: **SELECT * FROM TableA NATURAL JOIN TableB;**

- *Example:* If you have `Student(RollNo, Name)` and `Marks(RollNo, Subject, Grade)`, running `SELECT * FROM Student NATURAL JOIN Marks;` will automatically join the tables on the `RollNo` column, outputting a result with columns `(RollNo, Name, Subject, Grade)`.

29. How can we compare using null values?

Explain about logical connectives with examples.

In a database, a `NULL` value represents missing, unknown, or inapplicable data. It is not equivalent to zero or a blank space. Because its value is unknown, standard comparison operators (like `=`, `<`, `>`) cannot be used directly with `NULL`. Instead, SQL provides specific operators: `IS NULL` and `IS NOT NULL`.

When comparing with `NULL`, SQL uses **three-valued logic**: `TRUE`, `FALSE`, and `UNKNOWN`. Any standard comparison with `NULL` yields `UNKNOWN`.

Logical Connectives (AND, OR, NOT) with UNKNOWN:

- **AND:** Yields `TRUE` only if both operands are `TRUE`. If one is `FALSE`, it yields `FALSE`. If one is `TRUE` and the other is `UNKNOWN`, it yields `UNKNOWN`.
 - *Example:* `(TRUE AND UNKNOWN) → UNKNOWN`
- **OR:** Yields `TRUE` if at least one operand is `TRUE`. Yields `FALSE` only if both are `FALSE`.
 - *Example:* `(TRUE OR UNKNOWN) → TRUE`
- **NOT:** Reverses the boolean value.
 - *Example:* `NOT(UNKNOWN) → UNKNOWN`

30. Write about triggers in databases with example.

A **Trigger** is a special type of stored procedure in a database that automatically executes (or "fires") when a specific event occurs on a particular table or view. These events are typically DML operations: `INSERT`, `UPDATE`, or `DELETE`. Triggers are used to enforce business rules, maintain audit trails, and ensure data integrity.

Structure of a Trigger:

1. **Event:** The database operation that causes the trigger to fire (e.g., `BEFORE UPDATE`).
2. **Condition:** (Optional) A rule that determines whether the trigger action should execute.
3. **Action:** The SQL code that runs when the event and condition are met.

Example: Creating an audit trail to log salary changes.

SQL

```
CREATE TRIGGER Salary_Audit
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO Audit_Log (EmpID, OldSalary, NewSalary, ChangeDate)
        VALUES (OLD.EmpID, OLD.salary, NEW.salary, CURRENT_DATE);
    END IF;
END;
```

31. Explain the structure of basic form of an SQL query.

The basic structure of a standard SQL query consists of three primary clauses: `SELECT`, `FROM`, and `WHERE`.

1. **SELECT Clause:** Specifies the columns (attributes) you want to retrieve and display in the result set. It corresponds to the projection operation in relational algebra.
 - *Example:* `SELECT Name, Age`
2. **FROM Clause:** Specifies the table (or tables) from which the data should be retrieved. It corresponds to the cartesian product or join operations if multiple tables are listed.
 - *Example:* `FROM Student`

3. **WHERE Clause:** (Optional) Specifies the condition(s) that the rows must satisfy to be included in the result. It acts as a filter and corresponds to the selection operation in relational algebra.
 - Example: **WHERE Age > 18**

32. What is cursor in database? How can you create an user defined cursor to store all manager details in SQL?

A **Cursor** is a temporary work area created in the system memory when a SQL statement is executed. While SQL operates on sets of data (returning multiple rows at once), standard procedural programming languages need to process data row by row. A cursor acts as a pointer that allows you to iterate through a result set one row at a time.

Steps to create a user-defined cursor:

1. **DECLARE:** Define the cursor and the SQL query it will execute.
2. **OPEN:** Execute the query and populate the cursor.
3. **FETCH:** Retrieve the current row into variables and move the pointer to the next row.
4. **CLOSE:** Deactivate the cursor.
5. **DEALLOCATE:** Free the memory assigned to the cursor.

Example:

SQL

```
DECLARE @MgrID INT, @MgrName VARCHAR(50);
```

```
-- 1. Declare
```

```
DECLARE ManagerCursor CURSOR FOR
```

```
SELECT EmpID, Name FROM Employee WHERE Role = 'Manager';
```

```
-- 2. Open
```

```
OPEN ManagerCursor;
```

```
-- 3. Fetch
```

```
FETCH NEXT FROM ManagerCursor INTO @MgrID, @MgrName;
```

```
WHILE @@FETCH_STATUS = 0
```

```
BEGIN
```

```
    PRINT 'Manager Name: ' + @MgrName;
```

```
    FETCH NEXT FROM ManagerCursor INTO @MgrID, @MgrName;
```

```
END;
```

```
-- 4 & 5. Close and Deallocate
```

```
CLOSE ManagerCursor;
```

```
DEALLOCATE ManagerCursor;
```

33. Consider the given relations: ... Write Relational Algebra expression for the following queries.

Given relations: **STUDENT(SID, Name, Dept, Age)** **COURSE(CID, CName, Dept)**
ENROLL(SID, CID, Marks)

- Find names of all students:
 $\pi_{\{Name\}}(STUDENT)$
- Find names of students whose age is greater than 20:
 $\pi_{\{Name\}}(\sigma_{\{Age > 20\}}(STUDENT))$
- Find students enrolled in DBMS:
 $\pi_{\{Name\}}(STUDENT \bowtie (\sigma_{\{CName = 'DBMS'\}}(COURSE) \bowtie ENROLL))$
- Find students who are not enrolled in any course: This uses the set difference operator to subtract the IDs of enrolled students from all student IDs.
 $\pi_{\{Name\}}(STUDENT \bowtie (\pi_{\{SID\}}(STUDENT) - \pi_{\{SID\}}(ENROLL)))$

- Find students who are enrolled in all courses: This requires the division operator ($\div$). We divide the **ENROLL** combinations by all available **COURSE** IDs.

$$\pi_{\text{Name}}(\text{STUDENT} \bowtie (\pi_{\text{SID, CID}}(\text{ENROLL}) \div \pi_{\text{CID}}(\text{COURSE})))$$

34. What is redundancy? Explain with suitable example.

Data Redundancy is the unnecessary duplication of data within a database. It occurs when the same piece of information is stored in multiple places. Redundancy is a primary issue in flat-file systems and poorly designed databases.

Example: Consider a single table tracking employees and their departments:

Table: **Employee** (EmpID, EmpName, DeptID, DeptName, DeptLocation)

If 50 employees work in the "IT" department located in "Building A", the values "IT" and "Building A" are repeated 50 times in the database. This repetition wastes storage space and leads to maintenance issues.

35. What are the anomalies created by data redundancy? Explain.

Data redundancy leads to three severe operational problems known as anomalies:

- Update Anomaly:** Occurs when duplicate data is updated in one place but not in others, leading to data inconsistency. In the example above, if the IT department moves to "Building B", you must update 50 separate records. Missing even one creates conflicting data.
- Insertion Anomaly:** Occurs when certain attributes cannot be inserted into the database without the presence of other attributes. If a new department "HR" is created but has no employees yet, you cannot add it to the **Employee** table because **EmpID** (the primary key) cannot be null.
- Deletion Anomaly:** Occurs when deleting a record inadvertently causes the loss of other required data. If the only employee in the "Finance" department resigns and their record is deleted, the database completely loses all information about the existence of the "Finance" department.

36. Explain the classification of functional dependency.

A Functional Dependency (FD), denoted as $X \rightarrow Y$, means the value of attribute X uniquely determines the value of attribute Y. They are classified as:

- Trivial Dependency:** $X \rightarrow Y$ is trivial if Y is a subset of X. (e.g., {RollNo, Name} $\rightarrow$ Name). It's always true.
- Non-Trivial Dependency:** $X \rightarrow Y$ is non-trivial if Y is not a subset of X. (e.g., RollNo $\rightarrow$ Name).
- Completely Non-Trivial Dependency:** If the intersection of X and Y is completely empty ($X \cap Y = \emptyset$).
- Partial Dependency:** Occurs when a non-prime attribute depends on only a *part* of a composite primary key, rather than the whole key. This violates 2NF.
- Full Functional Dependency:** A non-prime attribute depends on the *entire* primary key, and removing any attribute from the key means the dependency no longer holds.
- Transitive Dependency:** Occurs when $A \rightarrow B$ and $B \rightarrow C$, therefore $A \rightarrow C$ (where B is not a candidate key). This violates 3NF.

37. Consider the relation schema R(A,B,C,D,E,F) and the functional dependencies... Find the candidate keys of this relation R?

(Note: Based on standard notation, interpreting the OCR typo $A \rightarrow B, C$ as $A \rightarrow B, C$ and $>DF$ as $\rightarrow DF$)

Given: Relation R(A, B, C, D, E, F)

FDs: {A $\rightarrow$ B, C $\rightarrow$ DF, AC $\rightarrow$ E, D $\rightarrow$ F}

To find candidate keys, we calculate the closure of attribute sets, looking for a set that can derive all attributes {A,B,C,D,E,F} without being reducible.

Let's test {A, C} because A and C do not appear on the right side of any dependency (they cannot be derived, so they must be part of the key).

- Closure of AC (AC^{+}):**
 - Start with $AC^{+} = \{A, C\}$

2. Using $A \rightarrow B$, add B: $AC^+ = \{A, B, C\}$
3. Using $C \rightarrow DF$, add D, F: $AC^+ = \{A, B, C, D, F\}$
4. Using $AC \rightarrow E$, add E: $AC^+ = \{A, B, C, D, E, F\}$

Since AC^+ covers all attributes in the relation, and neither A^+ nor C^+ can cover all attributes independently, **AC is the only candidate key.**

38. Define and explain 4NF with suitable example.

Fourth Normal Form (4NF) deals with handling multivalued dependencies. A relation is in 4NF if, and only if, it is already in Boyce-Codd Normal Form (BCNF), and it contains no non-trivial multivalued dependencies.

A multivalued dependency ($X \twoheadrightarrow Y$) exists when one attribute (X) uniquely determines a *set* of values for another attribute (Y), independent of other attributes in the table.

Example:

Suppose a **Student** can enroll in multiple **Courses** and participate in multiple **Hobbies**.

Table: (StudentID, Course, Hobby)

- Student 1 takes Math and Physics.
- Student 1 plays Tennis and Chess.

To store this, the table would look like:

StudentID	Course	Hobby
1	Math	Tennis
1	Math	Chess
1	Physics	Tennis
1	Physics	Chess

Here, **StudentID** determines multiple **Courses**, and independently determines multiple **Hobbies** ($\text{StudentID} \twoheadrightarrow \text{Course}$ and $\text{StudentID} \twoheadrightarrow \text{Hobby}$). This causes severe redundancy.

Solution (Decomposition to 4NF): Split the table into two:

1. **Student_Course**(StudentID, Course)
2. **Student_Hobby**(StudentID, Hobby)

39. Explain join dependencies with fifth normal form with an example.

Fifth Normal Form (5NF), also known as Project-Join Normal Form (PJNF), ensures that a table can be decomposed into smaller tables and then successfully rejoined (using Natural Join) without losing any original records or generating spurious (fake) records. This is governed by **Join Dependencies**. A relation is in 5NF if every join dependency in it is implied by the candidate keys.

Example:

Consider a table linking Suppliers, Parts, and Projects: **Supply**(Supplier, Part, Project).

Suppose the rule is: "If Supplier S supplies Part P, and Project J requires Part P, and Supplier S is contracted to Project J, then S *must* supply P to J."

If we decompose this into three tables:

1. **(Supplier, Part)**
2. **(Part, Project)**
3. **(Supplier, Project)**

Because of the cyclic join dependency, if we perform a natural join on these three smaller tables, we recreate the exact original table without generating invalid data.

Therefore, the original table should be decomposed into these three to reach 5NF.

40. What is meant by multivalued dependency? Explain with example.

As discussed in 4NF, a **Multivalued Dependency (MVD)** occurs in a table with at least three attributes (e.g., A, B, and C). It exists when a single value of attribute A uniquely determines a set of multiple values for attribute B, and independently determines a set of values for attribute C.

The formal notation is $A \twoheadrightarrow B$ (A multidetermines B).

Example:

Consider **Restaurant(Name, Pizza_Topping, Delivery_Area)**.

- "Pizza Hut" offers toppings: Pepperoni, Mushroom.
 - "Pizza Hut" delivers to areas: North, South.
- The toppings a restaurant offers are completely independent of the areas they deliver to. However, if stored in one table, you must create a Cartesian product of toppings and areas for "Pizza Hut", leading to massive redundancy. Here, $\text{Name} \twoheadrightarrow \text{Pizza_Topping}$ and $\text{Name} \twoheadrightarrow \text{Delivery_Area}$.

41 & 42. Consider the relation $R(v,w,x,y,z)$ with functional dependencies... Find the closure for $\{xy\}$.

Given: Relation $R(v, w, x, y, z)$ FDs: $\{z \rightarrow y, y \rightarrow z, x \rightarrow y, x \rightarrow v, vw \rightarrow x\}$.

We need to find the closure of the set $\{x, y\}$, denoted as $\{x, y\}^+$.

1. **Initialize:** Start with the attributes themselves. $\{x, y\}^+ = \{x, y\}$
2. **Apply $y \rightarrow z$:** Since y is in our set, we can add z. $\{x, y\}^+ = \{x, y, z\}$
3. **Apply $x \rightarrow v$:** Since x is in our set, we can add v. $\{x, y\}^+ = \{v, x, y, z\}$
4. **Check remaining FDs:**
 - $z \rightarrow y$: y is already in the set.
 - $x \rightarrow y$: y is already in the set.
 - $vw \rightarrow x$: We have v, but we do not have w. Therefore, we cannot use this dependency.

Result: The closure $\{x, y\}^+ = \{v, x, y, z\}$.

43. Define the terms in the context of file and storage management.

Seek time, access time, rotational latency.

These terms describe the physical delays associated with retrieving data from a mechanical Hard Disk Drive (HDD):

- **Seek Time:** The time it takes for the disk controller to physically move the read/write head arm to the specific track (cylinder) on the disk where the requested data is located.
- **Rotational Latency:** Once the head is on the correct track, this is the time spent waiting for the disk platter to rotate so that the desired sector aligns directly under the read/write head.
- **Access Time:** The total time elapsed between a data request being issued and the actual data transfer beginning. It is calculated as: **Access Time = Seek Time + Rotational Latency + Command Processing Time.**

44. Compare Primary indexing and clustering indexing in file management techniques.

Both are ordered indexing techniques, but they differ fundamentally in how the underlying data file is organized.

Feature	Primary Index	Clustering Index
Key Type	Built on the Primary Key of the table.	Built on a Non-Primary Key (but often a candidate for frequent searching).
Data Order	The physical data file is strictly sequentially ordered based on the primary key field.	The physical data file is sequentially ordered based on the clustering key field.

Uniqueness	The search key values are strictly unique.	The search key values do not have to be unique (e.g., clustering by Department Name).
Density	Can be a sparse index (one index entry per disk block) because the file is ordered and keys are unique.	Often sparse, pointing to the first block containing a record with that specific cluster key.
Quantity	A file can only have one Primary index.	A file can only have one Clustering index (because the physical data can only be sorted one way).

45. Disadvantages of B TREE over B+TREE.

B+ Trees are generally preferred for database indexing over standard B Trees due to several key disadvantages of the B Tree architecture:

1. **Data at Internal Nodes:** In a B Tree, search keys and actual data pointers (or records) are stored in internal nodes as well as leaf nodes. This takes up space, reducing the number of keys that can fit in a single disk block (lower fanout). A B+ Tree only stores data pointers at the leaf nodes, maximizing internal node capacity and keeping the tree shallower for faster lookups.
2. **Inefficient Range Queries:** Finding a range of values (e.g., **Salary BETWEEN 50k AND 100k**) in a B Tree requires traversing up and down the tree branches repeatedly. In a B+ Tree, all leaf nodes are linked together in a sequential linked list, making range queries incredibly fast (just traverse the list).
3. **Deletion Complexity:** Deleting a node from a B Tree can be highly complex if the key is located in an internal node, often requiring major structural adjustments. B+ Tree deletions are simpler since actual data is only deleted from the leaf level.

46. Consider the following relation: Shipping...

Given: **Shipping** (Shipname, Shiptype, Voyageid, Cargo, Port, Date) FDs: {ShipName → Shiptype; Voyageid → Shipname, Cargo; [cite_start] Shipname, Date → Voyageid, Port}

a) Identify the candidate keys. Let's find combinations that derive all attributes.

1. Check {Shipname, Date}⁺⁺:
 - Adds {Voyageid, Port} via 3rd FD. Set: {Shipname, Date, Voyageid, Port}
 - Adds {Shiptype} via 1st FD. Set: {Shipname, Date, Voyageid, Port, Shiptype}
 - Adds {Cargo} via 2nd FD. Set: {Shipname, Date, Voyageid, Port, Shiptype, Cargo}
 - **Result:** {Shipname, Date} is a candidate key.
2. Check {Voyageid, Date}⁺⁺:
 - Adds {Shipname, Cargo} via 2nd FD. Set: {Voyageid, Date, Shipname, Cargo}
 - Adds {Shiptype} via 1st FD. Set: {Voyageid, Date, Shipname, Cargo, Shiptype}
 - Adds {Port} via 3rd FD (since we have Shipname and Date). Set: {Voyageid, Date, Shipname, Cargo, Shiptype, Port}
 - **Result:** {Voyageid, Date} is a candidate key.

b) Normalize the relation until it is completely normalized. The table is not in 2NF because of partial dependencies: **Shipname** is part of a candidate key, but **ShipName → Shiptype**

Shiptype means **Shiptype** depends only on part of the key. *Decomposition to BCNF:*

- R_1(\underline{Shipname}, Shiptype) (Resolves ShipName → Shiptype)
- R_2(\underline{Voyageid}, Shipname, Cargo) (Resolves Voyageid → Shipname, Cargo)

- $R_3(\underline{\text{Shipname, Date}}, \text{Voyageid, Port})$ (Resolves the remaining attributes)

47. Define BCNF. How does it differ from 3 NF? Why is it considered as stronger than 3 NF?

- **Boyce-Codd Normal Form (BCNF):** A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$ present in the relation, X is a superkey. Simply put, "every determinant must be a candidate key."
- **Difference from 3NF:** 3NF has two conditions for $X \rightarrow Y$: either X is a superkey, OR Y is a prime attribute (part of a candidate key). BCNF removes this second lenient condition.
- **Why it is stronger:** BCNF is stronger because it catches edge-case anomalies that 3NF misses. Specifically, 3NF allows a non-key attribute to determine a prime attribute (part of a key). This overlapping key dependency can still cause redundancy. BCNF strictly forbids this, ensuring higher data integrity.

48. Discuss Armstrong's Axioms.

Armstrong's Axioms are a set of fundamental inference rules used to deduce all possible functional dependencies within a relational database. They are sound (they do not generate false dependencies) and complete (they can find all true dependencies).

The three primary axioms are:

1. **Axiom of Reflexivity:** If Y is a subset of X , then $X \rightarrow Y$. (Trivial dependency).
2. **Axiom of Augmentation:** If $X \rightarrow Y$, then for any attribute set Z , $XZ \rightarrow YZ$. (Adding the same attribute to both sides preserves the dependency).
3. **Axiom of Transitivity:** If $X \rightarrow Y$ and $Y \rightarrow Z$, then it logically follows that $X \rightarrow Z$.

From these, three secondary rules can be derived:

- **Union:** If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.
- **Decomposition:** If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.
- **Pseudotransitivity:** If $X \rightarrow Y$ and $YW \rightarrow Z$, then $XW \rightarrow Z$.

49. Explain, how secondary indexing can help in join operation.

A secondary index is an index constructed on a non-ordering attribute of a data file. During a join operation (particularly an Index Nested-Loop Join), secondary indexing dramatically reduces I/O operations and speeds up the query.

How it helps:

Suppose you are joining **Table A** and **Table B** on the condition **$A.ID = B.A_ID$** .

If there is no index, the DBMS must perform a block nested-loop join, scanning every block of **Table B** for *each* record in **Table A**. This is incredibly slow.

If a **secondary index** exists on **$B.A_ID$** :

1. The DBMS reads a tuple from **Table A**.
2. Instead of scanning all of **Table B**, it searches the secondary index using the **$A.ID$** value.
3. The index immediately provides the physical address pointers to the exact records in **Table B** that match the join condition.
4. The DBMS fetches only those specific blocks, bypassing a full table scan and making the join highly efficient.

50. Given below are two sets of FD's for a relation $R(A,B,C,D,E)$. Are they equivalent? $F=\{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$ and $G=\{A \rightarrow CD, E \rightarrow AH\}$

Two sets of functional dependencies (FDs) are equivalent if every FD in F can be derived from G (G covers F), and every FD in G can be derived from F (F covers G).

Step 1: Check if G covers F (Can we derive F using G):

- **$A \rightarrow C$:** In G , we have $A \rightarrow CD$. By the decomposition rule, $A \rightarrow C$. (True)
- **$AC \rightarrow D$:** In G , $A \rightarrow CD \Rightarrow A \rightarrow D$. By the augmentation rule, adding C gives $AC \rightarrow CD \Rightarrow AC \rightarrow D$. (True)

- **E → AD:** In G, $E \rightarrow AH \Rightarrow E \rightarrow A$. Since we also know from G that $A \rightarrow CD \Rightarrow A \rightarrow D$. By transitivity, $E \rightarrow A$ and $A \rightarrow D \Rightarrow E \rightarrow D$. By the union rule of $E \rightarrow A$ and $E \rightarrow D$, we get $E \rightarrow AD$. (True)
- **E → H:** In G, $E \rightarrow AH \Rightarrow E \rightarrow H$ by decomposition. (True)
- **Conclusion for Step 1:** G successfully covers F.

Step 2: Check if F covers G (Can we derive G using F?):

- **A → CD:** In F, we have $A \rightarrow C$. We need to find if $A \rightarrow D$. We know $AC \rightarrow D$. Since $A \rightarrow A$ (trivial) and $A \rightarrow C$, we can combine them to say $A \rightarrow AC$. Since $A \rightarrow AC$ and $AC \rightarrow D$, transitivity gives $A \rightarrow D$. Using the union rule on $A \rightarrow C$ and $A \rightarrow D$, we get $A \rightarrow CD$. (True)
- **E → AH:** In F, we have $E \rightarrow AD \Rightarrow E \rightarrow A$. We also have $E \rightarrow H$. Using the union rule, $E \rightarrow AH$. (True)
- **Conclusion for Step 2:** F successfully covers G.

Final Answer: Yes, the two sets of functional dependencies F and G are **equivalent**.

51. "All BCNF relations are in 3 NF but not vice versa." Prove the statement with example.

Proof of Statement:

- For a relation to be in **3NF**, every non-trivial functional dependency $X \rightarrow Y$ must satisfy at least one of two conditions:
 1. X is a superkey.
 2. Y is a prime attribute (part of a candidate key).
- For a relation to be in **BCNF**, every non-trivial functional dependency $X \rightarrow Y$ must satisfy the condition:
 1. X is a superkey.
- Because BCNF's strict requirement directly fulfills the first acceptable condition of 3NF, **every relation in BCNF is inherently in 3NF**. However, a relation can satisfy 3NF by utilizing the second condition (Y is a prime attribute) while failing the BCNF condition.

Counter-Example (3NF but NOT BCNF):

Consider a university relation **R(Student, Course, Instructor)** with the following rules:

1. A student can take multiple courses, but only has one instructor per course. $\Rightarrow \{Student, Course\} \rightarrow Instructor$
 2. An instructor teaches only one specific course. $\Rightarrow Instructor \rightarrow Course$
- **Candidate Keys:** {Student, Course} and {Student, Instructor}.
 - **Prime Attributes:** Student, Course, Instructor (all attributes are prime).
 - **Checking 3NF:** Both dependencies are valid in 3NF because the right side of both FDs are prime attributes.
 - **Checking BCNF:** The dependency $Instructor \rightarrow Course$ violates BCNF because **Instructor** alone is **not** a superkey (one instructor doesn't uniquely identify a student).
 - Therefore, the relation is in 3NF, but it is **not** in BCNF.

52. Consider insertion sequence- 2,3,5,11,17,19,23,29,31. Construct B+ Tree for the case where the number of pointers that will fit in one node is 4.

If a node can hold 4 pointers ($p = 4$), the maximum number of keys it can hold is $p - 1 = 3$. An overflow occurs when a node receives a 4th key. During a split, the middle key moves up to the parent node.

Step-by-Step Construction:

1. **Insert 2, 3, 5:** The root acts as a leaf node. [2, 3, 5]
2. **Insert 11 (Overflow):** Node becomes [2, 3, 5, 11]. Split it. Keep [2, 3] on the left, [5, 11] on the right. Copy the middle key (5) up to a new root.
 - Root: [5]
 - Leaves: [2, 3] → [5, 11]
3. **Insert 17:** Goes to the right leaf.
 - Leaves: [2, 3] → [5, 11, 17]

4. **Insert 19 (Overflow):** Right leaf becomes [5, 11, 17, 19]. Split it to [5, 11] and [17, 19]. Copy 17 up to the root.
 - Root: [5, 17]
 - Leaves: [2, 3] → [5, 11] → [17, 19]
5. **Insert 23:** Goes to the rightmost leaf.
 - Leaves: [2, 3] → [5, 11] → [17, 19, 23]
6. **Insert 29 (Overflow):** Rightmost leaf becomes [17, 19, 23, 29]. Split it to [17, 19] and [23, 29]. Copy 23 up to the root.
 - Root: [5, 17, 23]
 - Leaves: [2, 3] → [5, 11] → [17, 19] → [23, 29]
7. **Insert 31:** Goes to the rightmost leaf.

Final B+ Tree Structure:

- **Root Node:** [5, 17, 23]
- **Leaf Level (Linked List):** [2, 3] → [5, 11] → [17, 19] → [23, 29, 31]

54. Define ACID properties of a transaction in DBMS.

ACID properties ensure that database transactions are processed reliably:

1. **Atomicity ("All or Nothing"):** Guarantees that a transaction is treated as a single indivisible unit. Either all its operations are executed completely, or none of them are. If it fails midway, the database is rolled back to its initial state.
2. **Consistency:** Ensures that a transaction brings the database from one valid, consistent state to another. All predefined rules, constraints, and triggers must be successfully applied.
3. **Isolation:** Guarantees that concurrent execution of transactions leaves the database in the same state that would have been obtained if the transactions were executed sequentially. The intermediate states of a transaction are invisible to other transactions.
4. **Durability:** Once a transaction has been successfully committed, its changes are permanently recorded in the database, surviving even system crashes or power failures.

53. Draw the precedence graph for the schedule ... Test whether the schedule is conflict serializable or not?

S: {r₁(x), r₂(y), r₃(y), w₂(y), w₁(x), w₃(x), r₂(x), w₂(x)}

CONFLICT SERIALIZABILITY AND PRECEDENCE GRAPH ANALYSIS

SCHEDULE 1

$S_1 = w_1(A) w_2(A) r_3(B) w_3(B) r_1(B)$

1. IDENTIFY CONFLICTING OPERATIONS

- On Item A:
 - $w_1(A) \rightarrow w_2(A)$ (Write-Write) | Creates edge $T_1 \rightarrow T_2$
- On Item B:
 - $w_3(B) \rightarrow r_1(B)$ (Write-Read) | Creates edge $T_3 \rightarrow T_1$

2. PRECEDENCE GRAPH EDGES

- $T_3 \rightarrow T_1$
- $T_1 \rightarrow T_2$

3. SERIALIZABILITY TEST & ORDER

- Conflict Serializable? YES (No cycles).
- Correct Order: $T_3 \rightarrow T_1 \rightarrow T_2$ (Topological sort).

SCHEDULE 2

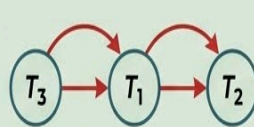
$S_2 = \{r_1(x); r_2(y); r_3(y); w_2(y); w_1(x); w_3(x); r_2(x); w_2(x)\}$

1. IDENTIFY CONFLICTING OPERATIONS

- On Item x:
 - $r_1(x) \rightarrow w_3(x)$ (R-W), $T_1 \rightarrow T_3$
 - $r_1(x) \rightarrow w_2(x)$ (R-W), $T_1 \rightarrow T_2$
 - $w_1(x) \rightarrow w_3(x)$ (W-W), duplicate $T_1 \rightarrow T_3$
 - $w_1(x) \rightarrow r_2(x)$ (W-R), duplicate $T_1 \rightarrow T_2$
 - $w_1(x) \rightarrow w_2(x)$ (W-W), duplicate $T_1 \rightarrow T_2$
 - $w_3(x) \rightarrow r_2(x)$ (W-R), $T_3 \rightarrow T_2$
 - $w_3(x) \rightarrow w_2(x)$ (W-W), duplicate $T_3 \rightarrow T_2$
- On Item y:
 - $r_3(y) \rightarrow w_2(y)$ (R-W), duplicate $T_3 \rightarrow T_2$

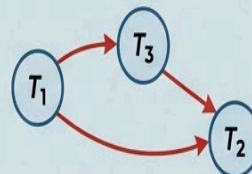
VISUAL PRECEDENCE GRAPHS

PRECEDENCE GRAPH FOR SCHEDULE 1



SERIAL ORDER: $T_3 \rightarrow T_1 \rightarrow T_2$

PRECEDENCE GRAPH FOR SCHEDULE 2



SERIAL ORDER: $T_1 \rightarrow T_3 \rightarrow T_2$

2. PRECEDENCE GRAPH EDGES

- $T_1 \rightarrow T_3$
- $T_1 \rightarrow T_2$
- $T_3 \rightarrow T_2$

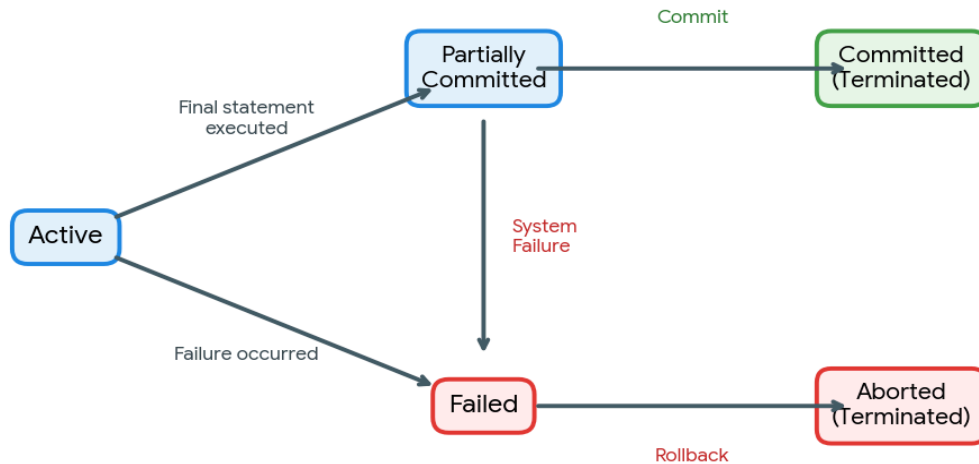
3. SERIALIZABILITY TEST & ORDER

- Conflict Serializable? YES (No cycles).
- Correct Order: $T_1 \rightarrow T_3 \rightarrow T_2$

55. Discuss different states of transaction with diagram.

A transaction in a database management system (DBMS) progresses through various states during its lifecycle to ensure data integrity and recovery.

- Active: The initial state where the transaction stays while executing its read and write operations.
- Partially Committed: Entered after the final execution statement is completed, but before changes are permanently saved to disk.
- Committed: Entered after successfully verifying all modifications, making the database updates permanent.
- Failed: Entered if a hardware, software, or logical error prevents normal execution from continuing.
- Aborted: Entered after the transaction is rolled back and the database is restored to its pre-transaction state.



56. Explain serial schedule with example.

A **Serial Schedule** is a schedule where transactions are executed completely one after the other, with no overlapping or interleaving of their operations. The next transaction only begins after the current transaction has fully committed or aborted.

Example:

Suppose we have two transactions, T₁ (transfer money) and T₂ (check balance). In a serial schedule, the CPU execution looks like this:

1. T₁ starts.
2. T₁ reads Account A.
3. T₁ writes to Account A.
4. T₁ commits.
5. T₂ starts (only after T₁ is entirely finished).
6. T₂ reads Account A.
7. T₂ commits.

Serial schedules guarantee data consistency but offer terrible performance and throughput because the system resources are locked by one transaction at a time.

57. When a schedule is called non-recoverable schedule? Explain.

A schedule is **non-recoverable** if a transaction T₂ reads a data item that was written by another transaction T₁, and T₂ commits *before* T₁ commits.

Explanation of the danger:

If T₂ reads uncommitted data (a "dirty read") from T₁, T₂ is depending on T₁'s success. If T₂ then commits, its changes become permanent. However, if T₁ subsequently fails and has to be aborted/rolled back, the data T₂ read is now invalid. Because T₂ already committed, the DBMS cannot roll it back. The database is now left in an inconsistent, permanently corrupted state.

58. Define Lock. Define two-phase locking.

- **Lock:** A lock is a concurrency control mechanism (a variable) associated with a specific data item. It determines whether a transaction has permission to read or write to that item. It prevents multiple transactions from modifying the same data simultaneously.
- **Two-Phase Locking (2PL):** A strict concurrency protocol that ensures conflict serializability. It requires every transaction to acquire and release locks in two distinct phases:
 1. **Growing Phase:** A transaction may acquire new locks on data items as needed, but it is strictly forbidden from releasing any locks.
 2. **Shrinking Phase:** Once the transaction releases its very first lock, it enters the shrinking phase. It may continue to release locks, but it is strictly forbidden from acquiring any new locks.

59. Differentiate between Serial Schedule and serializable schedule.

Explain different types of locks.

Serial vs. Serializable:

- **Serial Schedule:** Represents the *physical* execution order. Transactions run consecutively one by one without any time interleaving.
- **Serializable Schedule:** Represents the *logical* outcome. Transactions physically interleave operations concurrently to maximize CPU efficiency, but the concurrency control protocols (like locks) ensure that the final result in the database is 100% equivalent to as if they had run serially.

Types of Locks:

1. **Shared Lock (S-Lock):** If a transaction holds an S-lock on an item, it can only *read* the item. Crucially, multiple transactions can hold S-locks on the exact same data item simultaneously (concurrent reading is safe).
2. **Exclusive Lock (X-Lock):** If a transaction holds an X-lock, it can both *read and write* to the item. X-locks are strictly mutually exclusive. If one transaction holds an X-lock, no other transaction can acquire *any* lock (Shared or Exclusive) on that item until it is released.

60. Explain the concept of Hash File Organization with suitable diagram.

Hash File Organization uses a mathematical algorithm to compute the direct physical address (disk block) of a record based on its search key, bypassing the need for index lookups.

Concept:

1. **Hash Function (h):** A mathematical formula applied to a specific attribute (the hash key). E.g., $h(\text{Employee_ID}) = \text{Employee_ID} \% 10$.
2. **Buckets:** The storage space on the disk is divided into fixed-size units called buckets. The output of the hash function maps directly to a specific bucket address.
3. **Direct Access:** When searching for an Employee ID, the system runs the ID through the hash function, gets the bucket number, and jumps straight to that physical block on the disk.

(Visual Concept):

Plaintext

[Search Key: ID 104] ---> (Hash Function: ID % 5) ---> Output: 4

↓
V

Disk Storage:

[Bucket 0] -> Empty

[Bucket 1] -> Record 101

[Bucket 2] -> Empty

[Bucket 3] -> Record 103

[Bucket 4] -> [Record 104] (Direct retrieval!)

When a bucket gets full, a "collision" occurs, which is handled by creating "overflow buckets" linked via pointers (chaining).

61. Compare ordered indexing and hashing. How does multilevel indexing improve the efficiency of searching an index file?

Ordered Indexing vs. Hashing:

Feature	Ordered Indexing (e.g., B+ Trees)	Hashing
Best Query Type	Excellent for Range Queries (Salary BETWEEN 40k AND 60k, <, >).	Excellent for Exact Match Queries (ID = 101).

Search Time	Logarithmic time complexity $O(\log n)$.	Constant time complexity $O(1)$ ideally.
Data Ordering	Keys are stored in sorted sequential order.	Keys are scattered randomly based on the hash function.
Maintenance	Rebalancing the tree during inserts/deletes requires overhead.	Handling collisions and dynamic resizing (Extensible Hashing) requires overhead.

Multilevel Indexing:

When a database grows massive, the index file itself becomes too large to fit in the main memory (RAM). Scanning a massive index on the disk is slow.

Multilevel indexing solves this by creating an **"index for the index."** It treats the primary base index as a regular data file and creates a smaller, sparse secondary index on top of it. If that is still too large, a third level is added, creating a tree structure. This reduces disk I/O exponentially because the system only has to load a few small index blocks into memory to narrow down the search path, rather than loading the entire massive index file.

62. Given a block can hold either 3 records or 10 key pointers. A database contains n records, then how many blocks do we need to hold the data file and the dense index.

To find the total number of blocks, we calculate the blocks required for the data file and the index file separately.

1. Data File Blocks:

- Total number of records = n
- Capacity of one block = 3 records
- Blocks required to hold all data = $\lceil n / 3 \rceil$ (ceiling function, as partial blocks still consume a full block space).

2. Dense Index Blocks:

- By definition, a **dense index** contains one index entry (key pointer) for every *single record* in the data file.
- Total number of index pointers required = n
- Capacity of one block = 10 key pointers
- Blocks required to hold the dense index = $\lceil n / 10 \rceil$.

Total Blocks Required:

$$\{\text{Total Blocks}\} = \lceil n/3 \rceil + \lceil n/10 \rceil$$

63. The order of a leaf node in a B+ tree is the maximum number of (value, data record pointer) pairs it can hold. Given that the block size is 1K bytes, data record pointer is 7 bytes long, the value field is 9 bytes long and a block pointer is 6 bytes long, what is the order of the leaf node?

To find the order (n) of the leaf node, we must calculate how many (search key value, data record pointer) pairs can fit into a single disk block, alongside the single block pointer that links to the next leaf node.

Given Data:

- Block Size (B) = 1024 bytes (since 1K bytes = 1024 bytes)
- Data Record Pointer (P_r) = 7 bytes
- Search Key Value (V) = 9 bytes
- Block Pointer (P) = 6 bytes

Formula for a Leaf Node:

A leaf node in a B+ Tree holds n pairs of values and record pointers, plus exactly one block pointer pointing to the next sibling leaf node.

$$n \times (V + P_r) + P \leq B$$

Calculation:

$$n \times (9 + 7) + 6 \leq 1024$$

$$n \times (16) + 6 \leq 1024$$

$$16n \leq 1024 - 6$$

$$16n \leq 1018$$

$$n \leq \{1018 / \{16\}$$

$$n \leq 63.625$$

Since the number of pairs must be a whole integer, we take the floor of the result.

Answer: The order of the leaf node is **63**.

64. What is referential integrity constraints? How it is related to foreign key. Explain with example.

- **Referential Integrity Constraint:** This is a database rule ensuring that the relationships between tables remain logically consistent. It dictates that any value appearing in a designated foreign key column must either exactly match a value existing in the primary key column of the referenced parent table, or it must be explicitly **NULL**. This prevents "orphan" records.
- **Relation to Foreign Key:** A Foreign Key is the actual structural mechanism used to enforce referential integrity. When you define a foreign key, the DBMS automatically monitors data manipulation to ensure referential integrity is never violated.

Example:

Consider two tables: **Department** and **Employee**.

- **Department**(Dept_ID [Primary Key], Dept_Name)
- **Employee**(Emp_ID [Primary Key], Emp_Name, Dept_ID [Foreign Key])

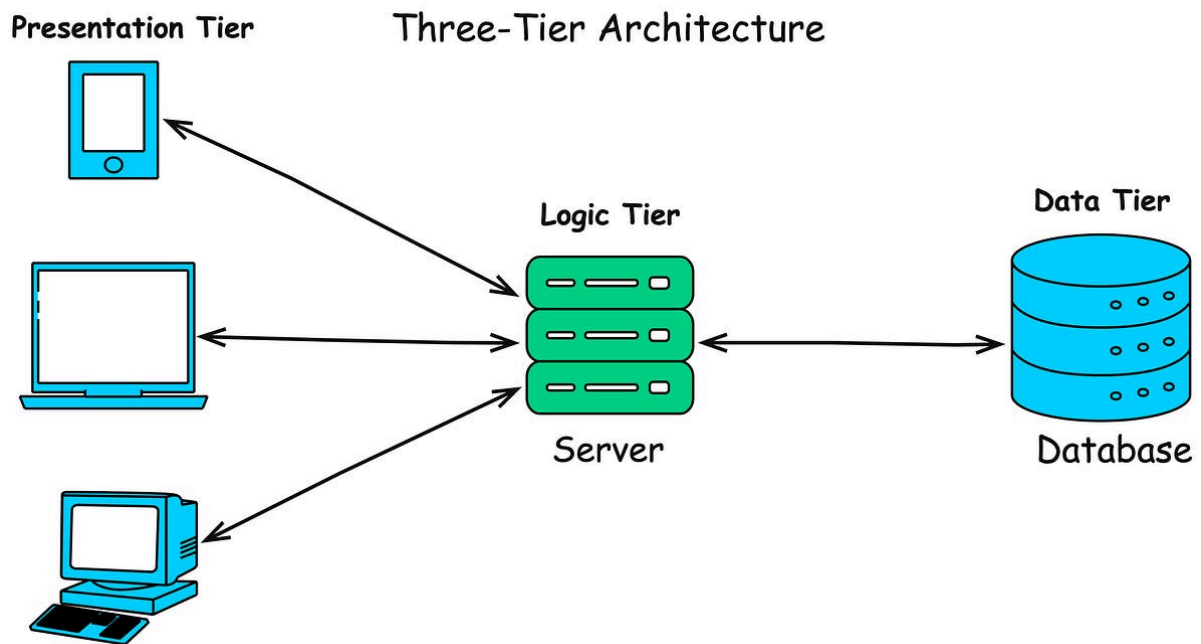
If you try to insert a new employee (105, "Alice", 99) into the **Employee** table, the DBMS will check the **Department** table. If **Dept_ID 99** does not exist in the **Department** table, the database will reject the insertion. This enforces referential integrity.

65. Discuss Three Tier Database Architecture with diagram.

The Three-Tier Architecture divides the system into three distinct logical layers, separating the user interface from the data storage and business logic.

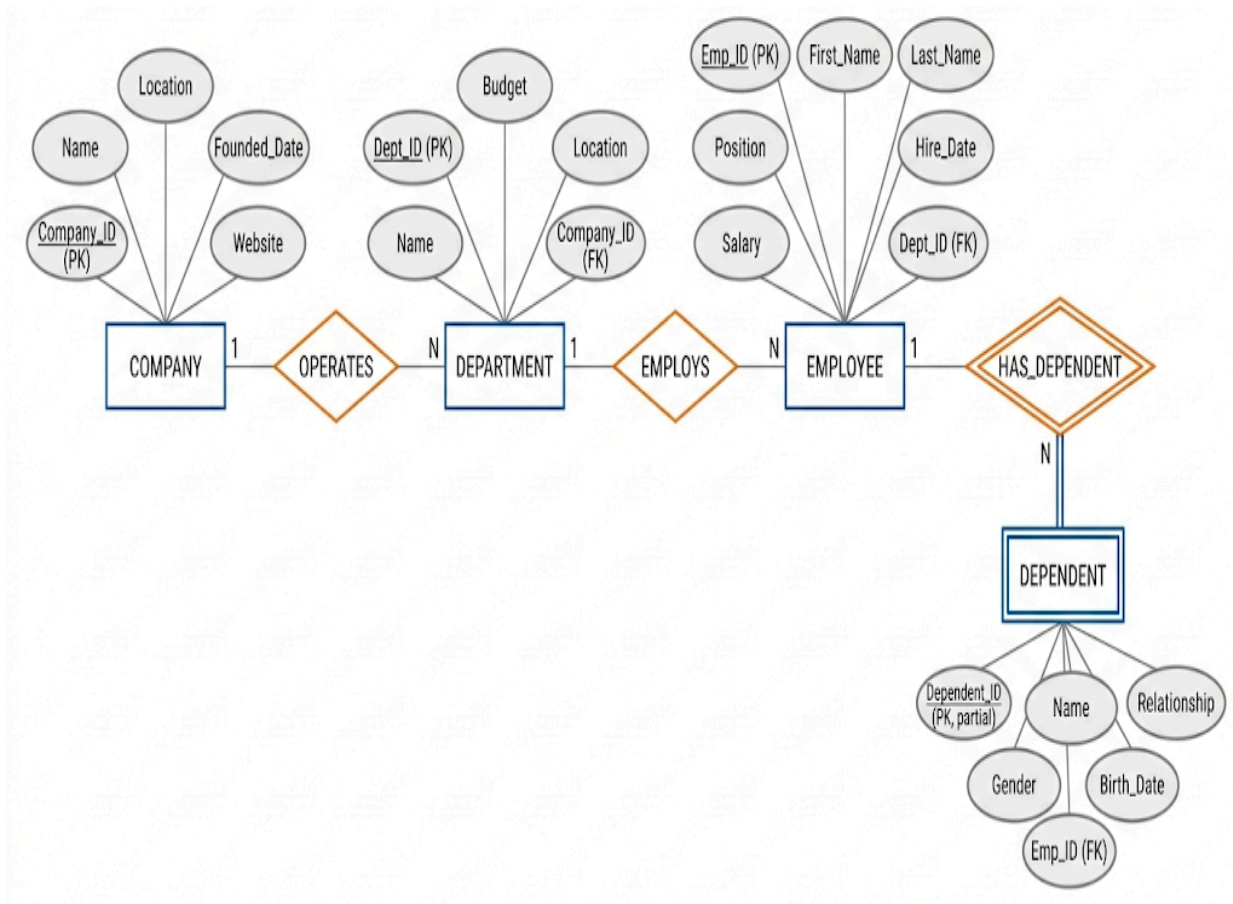
1. **Presentation Tier (Client/User Interface):** This is the topmost level where end-users interact with the system (e.g., a web browser or mobile app). It displays information and collects user inputs. It does not communicate directly with the database.
2. **Application Tier (Business Logic):** The middle layer (often an application server). It receives data from the presentation tier, processes it using specific business rules, and determines what data is needed from the database. It acts as a secure intermediary.
3. **Database Tier (Data Server):** The bottom layer containing the actual Database Management System (DBMS). It executes SQL queries received from the Application Tier and securely stores the physical data.

Diagram Layout :



66. Create ER diagram for each of the followings

1. Each company operates four departments and each department belongs to one company.
2. Each department in part (a) employs one or more employees and each employee works for one department.
3. Each of the employees in part(b) may or may not have one or more dependents.



67. What are the differences between natural join and cross product of two relations? Explain with example.

- **Cross Product (Cartesian Product):** This operation pairs every single row from the first table with every single row from the second table, regardless of whether the data makes sense together. It does not require tables to share common columns.
- **Natural Join:** This operation intelligently joins tables by automatically matching rows that have the same values in all identically named columns across both tables. Crucially, it eliminates the duplicate columns in the final output.

Example:

Suppose Table **A(X, Y)** has rows: (1, 10), (2, 20)

Suppose Table **B(Y, Z)** has rows: (10, "Apple"), (30, "Banana")

- **Cross Product ($A \bowtie B$):** Yields 4 rows with redundant columns.
(1, 10, 10, "Apple"), (1, 10, 30, "Banana"), (2, 20, 10, "Apple"), (2, 20, 30, "Banana").
- **Natural Join ($A \bowtie B$):** Looks for matches on column **Y**. Only the first row matches (**Y=10**). It drops the duplicate **Y** column.
Yields 1 row: (1, 10, "Apple").

68. Discuss each of followings- (a) Natural Join b) equi-join c) self join d) inner-join e) outer join

- **a) Natural Join:** An automated join that matches rows based on equality across all identically named columns in both tables. It automatically removes the redundant joining columns from the final result set.

- **b) Equi-Join:** A specific type of condition join (Theta join) where the operator used is strictly the equality sign (=). Unlike natural joins, it does *not* automatically remove duplicate columns; the user must specify them in the SELECT clause if they want them removed.
- **c) Self Join:** A regular join where a table is joined with itself. This requires using table aliases (e.g., **FROM Employee E1, Employee E2**) to treat the single table as two separate entities for the duration of the query. Used for hierarchical data (e.g., finding an employee's manager in the same table).
- **d) Inner-Join:** The default type of join. It returns *only* the records that have matching values in both tables based on the join condition. If a row in Table A has no corresponding match in Table B, it is excluded entirely. Natural and Equi-joins are subsets of inner joins.
- **e) Outer Join:** Unlike an inner join, an outer join returns all the matching rows *plus* the non-matching rows from one or both tables. The missing data on the non-matching side is filled with **NULL** values. It is divided into Left Outer Join, Right Outer Join, and Full Outer Join.

69. Consider the following relation schema... Write the SQL queries for the following.

Schema Given:

- **Works(Pname, Cname, salary)**
- **Lives(Pname, Street, City)**
- **Located_in (Cname, city)**
- **Manager(Pname, Mgrname)**

i) Find the names of all persons who live in the city Bangalore.

SQL

```
SELECT Pname
FROM Lives
WHERE City = 'Bangalore';
```

ii) Retrieve the names of all person of "Infosys" whose salary is between Rs.50000...

(Note: The question paper text cuts off at "Rs.50000". Assuming it implies a salary strictly greater than or equal to 50000 based on standard academic question phrasing).

SQL

```
SELECT Pname
FROM Works
WHERE Cname = 'Infosys' AND salary >= 50000;
```

iii) Find the names of all persons who lives and work in the same city

SQL

```
SELECT W.Pname
FROM Works W
JOIN Lives L ON W.Pname = L.Pname
JOIN Located_in Loc ON W.Cname = Loc.Cname
WHERE L.City = Loc.city;
```

iv) List the names of the people who work for "Tech M" along with the cities they live in.

SQL

```
SELECT W.Pname, L.City  
FROM Works W  
JOIN Lives L ON W.Pname = L.Pname  
WHERE W.Cname = 'Tech M';
```

v) Find the average salary of "Infosys" persons

SQL

```
SELECT AVG(salary)  
FROM Works  
WHERE Cname = 'Infosys';
```